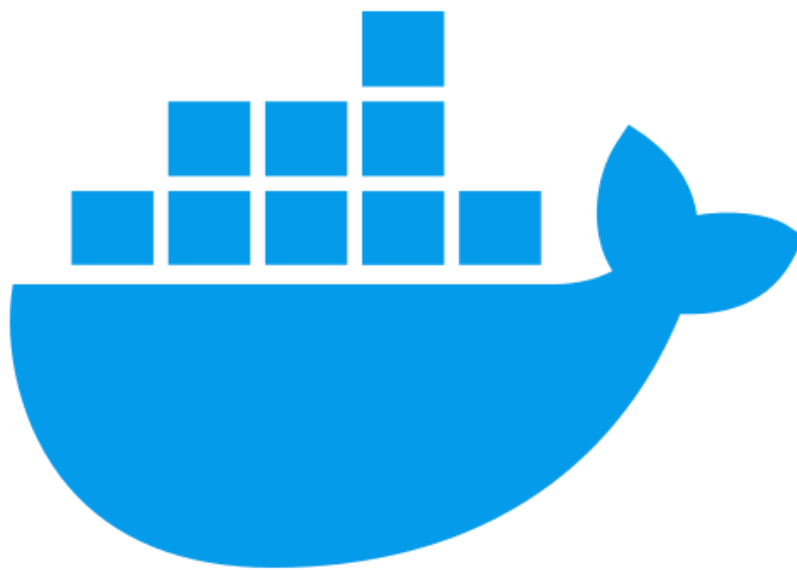

SWAP P3: SEGURIDAD (certificados SSL)

ÁLVARO GARCÍA ZAFRA



TIEMPO

TÍTULO ↕

DURACIÓN ↕

• SWAP P3

10:06:46

BÁSICAS

Tareas Básicas - B1: Preparación del Entorno de Trabajo

En esta parte se han creado los directorios de trabajo, tanto los que contendrán los archivos para los servidores como el que contendrá los certificados.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ mkdir P3-algarza04-certificados
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ mkdir P3-algarza04-apache
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ mkdir P3-algarza04-nginx
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ cp -r ../P1/P1-apache/web_algarza04/ P3-algarza04-apache/
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ cp -r ../P1/P1-apache/web_algarza04/ P3-algarza04-nginx/
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ tree
.
├── P3-algarza04-apache
│   └── web_algarza04
│       └── index.php
├── P3-algarza04-certificados
├── P3-algarza04-nginx
│   └── web_algarza04
│       └── index.php
└── SWAP_P3_guion.pdf
```

El directorio apache contendrá los archivos de configuración de apache, su Dockerfile y el index.php que mostrará la ip de cada servidor web y el de nginx tiene el mismo contenido que este último, por otro lado tenemos el directorio certificados, en él se almacenarán todos los certificados que iremos creando a lo largo de esta práctica.

Tareas Básicas - B2: Creación de certificados SSL

En primer lugar he comprobado si openssl, que es la herramienta que usaremos para crear el certificado, está instalado.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3/P3-algarza04-certificados$ openssl version
OpenSSL 3.0.13 30 Jan 2024 (Library: OpenSSL 3.0.13 30 Jan 2024)
```

Una vez comprobado esto me ayudé de la ia para saber como se debe crear una clave privada y un certificado autofirmado.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3/P3-algarza04-certificados$ openssl genrsa -out
certificado_algarza04.key 2048
```

En este paso creamos la clave privada con el nombre indicado, encriptado con RSA y con una longitud de 2048 bits.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3/P3-algarza04-certificados$ openssl req -new -x509
-key certificado_algarza04.key -out certificado_algarza04.crt -days 365
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:ES
State or Province Name (full name) [Some-State]:Granada
Locality Name (eg, city) []:Granada
Organization Name (eg, company) [Internet Widgits Pty Ltd]:SWAP
Organizational Unit Name (eg, section) []:Práctica 3
Common Name (e.g. server FQDN or YOUR name) []:Álvaro García Zafra
Email Address []:algarza04@correo.ugr.es
```

Una vez hemos generado la clave creamos el certificado con el tipo X509 (es el estándar de SSL), usando la clave privada que acabamos de generar y con una validez de 365 días. Además le he establecido el nombre que me indicaba el guión y los datos requeridos los he rellenado con las instrucciones que se indican en el guión.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3/P3-algarza04-certificados$ ls
certificado_algarza04.crt  certificado_algarza04.key
```

Como podemos ver tenemos ambos archivos creados en nuestro directorio certificados.

Tareas Básicas - B3: Configuración de Apache con SSL

En esta tarea vamos a configurar los servidores web apache para que estos puedan atender peticiones HTTPS usando certificados SSL autofirmados.

En primer lugar he creado el archivo de configuración de Apache que se va a encargar de habilitar SSL, este escuchará las peticiones HTTPS e incluye el certificado creado anteriormente.

```
# Definimos el VirtualHost que escuchará en el puerto 443
<VirtualHost *:443>

    # Directorio raíz donde se encuentran los archivos que se van a servir
    DocumentRoot /var/www/html

    # Activamos SSL
    SSLEngine on

    # Indicamos donde se encuentra el SSL público
    SSLCertificateFile /etc/apache2/ssl/certificado_algarza04.crt

    # Indicamos donde se encuentra la clave privada
    SSLCertificateKeyFile /etc/apache2/ssl/certificado_algarza04.key

</VirtualHost>
```

Una vez hemos hecho esto hemos pasado a la creación del DockerFile de Apache, para esto he partido del que creamos en la práctica 1 y le he aplicado los cambios convenientes (añadir los módulos necesarios para soportar y activar SSL, añadirle el nuevo archivo de configuración y exponer el puerto 443 para el tráfico HTTPS).

```
# El contenedor se crea a partir de la última imagen Debian
FROM debian:latest

# Indica que no se solicitará interacción con el usuario durante la instalación de paquetes
ENV DEBIAN_FRONTEND=noninteractive

# Instalar las dependencias necesarias
# (servidor web Apache, PHP, paquete de PHP para Apache y la herramienta ping, además limpia la cache)
RUN apt update && apt install -y apache2 php libapache2-mod-php iputils-ping && \
    rm -rf /var/lib/apt/lists/*

# Copia el .php al directorio de Apache que contiene los ficheros html
COPY P3-algarza04-apache/web_algarza04/index.php /var/www/html/

# Instalar módulo y habilitar sitio SSL por defecto y crear directorio para certificados
RUN a2enmod ssl && a2ensite default-ssl && mkdir /etc/apache2/ssl

# Copiar certificado y clave privada
COPY P3-algarza04-certificados/certificado_algarza04.crt /etc/apache2/ssl/certificado_algarza04.crt
COPY P3-algarza04-certificados/certificado_algarza04.key /etc/apache2/ssl/certificado_algarza04.key

# Configurar los permisos adecuados
RUN chmod 600 /etc/apache2/ssl/certificado_algarza04.crt

# Incluir la configuración SSL
COPY P3-algarza04-apache/algarza04-apache-ssl.conf /etc/apache2/sites-available/algarza04-apache-ssl.conf

# Exponer el puerto HTTPS
EXPOSE 443

# Inicia el servidor Apache en primer plano y mantiene el contenedor en ejecución(tail -f /dev/null)
CMD apache2ctl -D FOREGROUND && tail -f /dev/null
```

Posteriormente he creado una imagen de prueba para ver si el Dockerfile estaba creado correctamente. Al hacer esto he podido ver que no me funcionaba correctamente debido a que este no permite hacer COPY a los archivos que se encuentren fuera del contexto de construcción.

Para solucionar esto he ejecutado el DockerFile desde la raíz del directorio (añadiendo en que directorio se encuentra, es decir, *P3-algarza04-apache/DockerfileApache_algarza04*). Además le he añadido los directorios donde se encuentra cada archivo (*P3-algarza04-apache/web...*), para que sepa la localización de cada archivo.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker build -t apache-p3-image -f P3-algarza04-apache/DockerfileApache_algarza04 .
[+] Building 0.5s (13/13) FINISHED
                                docker:default
```

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker images
REPOSITORY          TAG          IMAGE ID          CREATED           SIZE
apache-p3-image     latest      a4a968056e49     37 seconds ago   265MB
```

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker run -d -p 8090:443 --name apache-p3 apache-p3-image:latest
11079d5224df25f0a69772797235f3b51f09d639877b4eafa3de1addd7be8906
```

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
NAMES
11079d5224df   apache-p3:latest  "/bin/sh -c 'apache2..."  15 seconds ago  Up 15 seconds  0.0.0.0:8090->443/tcp, [::]:8090->443/tcp
apache-p3

alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker exec -it apache-p3 /bin/bash
root@11079d5224df:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sy
```

Como podemos ver se ha levantado correctamente el contenedor con la imagen que hemos creado y nos deja acceder a él, por lo que parece que los archivos están correctamente implementados.

Tareas Básicas - B4: Configuración de Nginx con SSL

En esta tarea he implementado el archivo de configuración y el Dockerfile de Nginx.

En primer lugar he creado el archivo de configuración que se encargará de que Nginx maneje el tráfico HTTPS, para esto he copiado el .conf que creamos en la práctica anterior y le he añadido los cambios necesarios para que escuche HTTPS en lugar de HTTP y que aplique los certificados que hemos creado.

```
# define el usuario que ejecuta el proceso de nginx
user nginx;
# Configura el número de procesos de trabajo auto(en función de los núcleos CPU)
worker_processes auto;
# Especifica el archivo de registro de errores y el PID del proceso
error_log /var/log/nginx/error.log;
pid /var/run/nginx.pid;

events {
    # numero máximo de conexiones simultáneas
    worker_connections 1024;
}

http {
    # Define un grupo de servidores backend llamado "backend_algarza04".
    # Serán los servidores controlados por el balanceador de carga
    upstream backend_algarza04 {
        server 192.168.10.2;
        server 192.168.10.3;
        server 192.168.10.4;
        server 192.168.10.5;
        server 192.168.10.6;
        server 192.168.10.7;
        server 192.168.10.8;
        server 192.168.10.9;
    }

    server {
        # escucha en el puerto 443 y soporta SSL
        listen 443 ssl;

        # Especifica la ubicación del certificado SSL y su clave privada
        ssl_certificate /etc/nginx/ssl/certificado_algarza04.crt;
        ssl_certificate_key /etc/nginx/ssl/certificado_algarza04.key;

        location / {
            # Redirige las solicitudes al grupo de servidores creado anteriormente
            proxy_pass http://backend_algarza04;

            # reenvia las cookies y oculta el encabezado de la cookie
            proxy_set_header Cookie $http_cookie;
            proxy_hide_header Set-Cookie;
        }
    }
}
```

Posteriormente he copiado el Dockerfile de la práctica 2 y le he añadido los parámetros que se indican en el guión. Estos se encargarán de que se copie el certificado SSL y la clave privada al contenedor, le incluirán el archivo de configuración y abrirán el puerto 443(HTTPS).

```
# Crea el contenedor a partir de la última imagen de Nginx
FROM nginx:latest

# Crear directorio para SSL
RUN mkdir -p /etc/nginx/ssl

# Copiar certificados SSL al contenedor
COPY P3-algarza04-certificados/certificado_algarza04.crt /etc/nginx/ssl/certificado_algarza04.crt
COPY P3-algarza04-certificados/certificado_algarza04.key /etc/nginx/ssl/certificado_algarza04.key

# Incluir configuración de Nginx para SSL
COPY P3-algarza04-nginx/algarza04-nginx-ssl.conf /etc/nginx/nginx.conf

# Expose el puerto 443(HTTPS)
EXPOSE 443

# Inicia el servidor de Nginx en primer plano y lo mantiene en ejecución(tail -f /dev/null)
CMD nginx -g "daemon off;" && tail -f /dev/null
```

Para comprobar si la implementación de ambos archivos es correcta he realizado la misma comprobación que en el apartado de apache.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker build -t nginx-p3-image -f P3-algarza04-nginx/DockerfileNginx_algarza04 .
[+] Building 1.4s (10/10) FINISHED
```

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx-p3-image	latest	bf85aff1799c	About an hour ago	192MB

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker run -d -p 8090:443 --name nginx-p3 nginx-p3-image:latest
363b60517cf73a118a91e3a04fe3be5bb93a6b129eb87bfc13a390fdd838b0a6
```

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
363b60517cf7	nginx-p3-image:latest	"/docker-entrypoint..."	16 seconds ago	Up 16 seconds	80/tcp, 0.0.0.0:8090->443/tcp, [::]:8090->443/tcp
nginx-p3					

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker exec -it nginx-p3 /bin/bash
root@363b60517cf7:/# ls
bin dev docker-entrypoint.sh home lib64 mnt proc run srv t
boot docker-entrypoint.d etc lib media opt root sbin sys u
```

Como podemos ver se ha levantado correctamente el contenedor con la imagen que hemos creado y nos deja acceder a él, por lo que parece que los archivos están correctamente implementados. A pesar de que indique que el puerto 80 está abierto no escuchará por él, esto se debe a que nginx abre el puerto 80 por defecto pero al no tener el .conf con la orden listen 80 no escuchará por él.

Tareas Básicas - B5: Docker Compose y Despliegue de la Granja Web con SSL

Una vez creados todos los archivos, y tras realizar las pruebas para corroborar que estaban implementados correctamente, podemos pasar a crear el docker-compose que levantará el entorno.

A continuación podemos ver el código del docker-compose (de las redes apache solo aparece web1 ya que en las otras 7 solo aumentan en uno en su nombre, su puerto y la ip):

```
# inicialización de las redes web y servicios
networks:
  red_web:
    name: red_web
    ipam:
      config:
        - subnet: 192.168.10.0/24
  red_servicios:
    name: red_servicios
    ipam:
      config:
        - subnet: 192.168.20.0/24
```

```
# inicialización del contenedores web1:
# -es creado a partir del Dockerfile de apache
# -se le asigna su nombre e imagen
# -se asigna un puerto (HTTPS)
# -se asignan los volúmenes (dir web que posee el php y dir que posee los certificados)
# -se asignan las redes (red_web y red_servicios) con su ip estática correspondiente
▷ Run Service
web1:
  build:
    context: .
    dockerfile: P3-algarza04-apache/DockerfileApache_algarza04
  image: algarza04-apache-image:p3
  container_name: web1
  ports:
    - 8081:443
  volumes:
    - ./P3-algarza04-apache/web_algarza04:/var/www/html
    - ./P3-algarza04-certificados:/etc/apache2/ssl/
  networks:
    red_web:
      ipv4_address: 192.168.10.2
    red_servicios:
      ipv4_address: 192.168.20.2
```

```
# inicialización del balanceador-nginx-ssl:
# -se crea a partir del Dockerfile de nginx
# -se le asigna su nombre e imagen
# -se asigna un puerto (asociado al puerto HTTPS del contenedor, ya que servirá las peticiones de forma segura)
# -se asignan los volúmenes (dir que posee los certificados y dir que posee el archivo de configuración del balanceador)
# -se asigna la red_web con su ip estática correspondiente
# -se asigna la dependencia de los contenedores web1-8 (ya que el balanceador no puede iniciar sin ellos inicializados)
▷ Run Service
balanceador-nginx-ssl:
  build:
    context: .
    dockerfile: P3-algarza04-nginx/DockerfileNginx_algarza04
  image: algarza04-nginx-image:p3
  container_name: balanceador-nginx
  ports:
    - 8090:443
  volumes:
    - ./P3-algarza04-certificados:/etc/nginx/ssl/
    - ./P3-algarza04-nginx/algarza04-nginx-ssl.conf:/etc/nginx/nginx.conf
  networks:
    red_web:
      ipv4_address: 192.168.10.50
  depends_on:
    - web1
    - web2
    - web3
    - web4
    - web5
    - web6
    - web7
    - web8
```

A continuación vamos a pasar a comprobar si el docker-compose está bien implementado, si es correcto debería arrancar todos los servicios.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker-compose up -d
Creating network "red_web" with the default driver
Creating network "red_servicios" with the default driver
Creating web4 ... done
Creating web3 ... done
Creating web1 ... done
Creating web7 ... done
Creating web6 ... done
Creating web8 ... done
Creating web2 ... done
Creating web5 ... done
Creating balanceador-nginx ... done
```

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
52630f8114a8	algarza04-nginx-image:p3	balanceador-nginx	"/docker-entrypoint..."	18 seconds ago	Up 17 seconds	80/tcp, 0.0.0.0:8090->443/tcp, [::]:8090->443/tcp
45cb3d4f20f5	algarza04-apache-image:p3	web2	"/bin/sh -c 'apache2..."	18 seconds ago	Up 18 seconds	0.0.0.0:8082->443/tcp, [::]:8082->443/tcp
899b83be99cd	algarza04-apache-image:p3	web5	"/bin/sh -c 'apache2..."	18 seconds ago	Up 18 seconds	0.0.0.0:8085->443/tcp, [::]:8085->443/tcp
a9d270eac4a3	algarza04-apache-image:p3	web8	"/bin/sh -c 'apache2..."	18 seconds ago	Up 18 seconds	0.0.0.0:8088->443/tcp, [::]:8088->443/tcp
a2ff63691999	algarza04-apache-image:p3	web6	"/bin/sh -c 'apache2..."	18 seconds ago	Up 18 seconds	0.0.0.0:8086->443/tcp, [::]:8086->443/tcp
6080417ace2e	algarza04-apache-image:p3	web1	"/bin/sh -c 'apache2..."	18 seconds ago	Up 18 seconds	0.0.0.0:8081->443/tcp, [::]:8081->443/tcp
2d52954e7ca8	algarza04-apache-image:p3	web7	"/bin/sh -c 'apache2..."	18 seconds ago	Up 18 seconds	0.0.0.0:8087->443/tcp, [::]:8087->443/tcp
41693ca04f40	algarza04-apache-image:p3	web3	"/bin/sh -c 'apache2..."	18 seconds ago	Up 18 seconds	0.0.0.0:8083->443/tcp, [::]:8083->443/tcp
8867cbf808b4	algarza04-apache-image:p3	web4	"/bin/sh -c 'apache2..."	18 seconds ago	Up 18 seconds	0.0.0.0:8084->443/tcp, [::]:8084->443/tcp

Como podemos ver las redes se han creado correctamente y los servidores y el balanceador se han levantado correctamente, lo que significa que la implementación ha sido correcta.

Tareas Básicas - B6: Verificación del escenario SSL

En este apartado vamos a comprobar si nos deja acceder a cada uno de los contenedores y si el balanceador funciona correctamente y tiene un correcto funcionamiento del entorno SSL.

En primer lugar vamos a ver si nos deja acceder a todos ellos, lo he realizado uno a uno usando `curl -k https://...` (la opción `-k` nos permite ignorar errores en la validación del certificado).

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8081
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.20.2
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8082
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.20.3
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8083
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.20.4
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8084
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.20.5
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8085
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.20.6
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8086
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.20.7
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8087
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.20.8
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8088
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.20.9
```

Nos ha dejado acceder a todos por lo cuál es correcto.

El siguiente paso ha sido comprobar si el balanceador actúa de manera correcta, como podemos ver los está sirviendo en el orden correcto(round-robin).

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8090
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.10.2
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8090
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.10.3
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8090
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.10.4
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8090
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.10.5
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8090
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.10.6
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8090
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.10.7
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8090
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.10.8
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k https://localhost:8090
<h1>SWAP - "Álvaro García Zafra" </h1><br>Dirección IP del servidor: 192.168.10.9
```

Para que al acceder a la dirección del balanceador no me aparezca como no seguro he añadido el certificado a mi buscador.(No he conseguido que en mi buscador brave me deje de salir, sin embargo no es un problema que aparezca ya que se debe a que no está validando el certificado bien, aún así este funciona correctamente).



AVANZADAS

Tareas Avanzadas - A1: Exploraciones Avanzadas de creación de certificados SSL

- En primer lugar crearemos los dir para ir generando los certificados y claves privadas
 - `mkdir raizCA subCA servidor`
- Una vez que las tenemos vamos a crear el certificado raíz (CA):
 - Creamos su clave privada RSA, le damos una longitud de 4096 bits y le damos permiso de lectura y escritura para mi usuario: `openssl genrsa -out raizCA/ca-raiz.key 4096 chmod 600 subCA/subCA.key`.
 - Una vez generada la clave privada generamos el certificado autofirmado, el siguiente comando genera un certificado X509 valido por 10 años, con la firma sha256 y a partir de la clave privada creada anteriormente:

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicass/P3/P3-algarza04-certificados/miCA$ openssl req -x509 -new -nodes -key raizCA/ca-raiz.key -sha256 -days 3650 -out raizCA/ca-raiz.crt -subj "/C=ES/ST=Granada/L=Granada/O=SWAP/CN=CA_Raiz"
```

- Una vez hemos terminado lo anterior vamos a pasar a al creación de la subCA:
 - Creamos una clave privada siguiendo los mismos pasos del apartado anterior:

```
openssl genrsa -out subCA/subCA.key 4096
chmod 600 subCA/subCA.key
```

- Generamos un CSR a partir de la clave privada anterior que posteriormente firmará CARaiz, creando así un certificado válido:

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicass/P3/P3-algarza04-certificados/miCA$ openssl req -new -key subCA/subCA.key -out subCA/subCA.csr -subj "/C=ES/ST=Granada/L=Granada/O=SWAP/CN=Sub_CA"
```

- El siguiente paso es crear un archivo de extensiones, esto servirá para que no nos de problemas en el navegador, ya que sin esto no lo reconocerá como válido:

```
# Define que este certificado es de una CA con una cadena de certificados limitada (pathlen:0).
basicConstraints = critical, CA:TRUE, pathlen:0

# Permite firmar certificados, listas de revocación y datos digitales.
keyUsage = critical, digitalSignature, cRLSign, keyCertSign

# Identifica la clave pública del certificado y vincula con la CA emisora.
subjectKeyIdentifier = hash
authorityKeyIdentifier = keyid,issuer
```

- El último paso en este apartado es, como he dicho anteriormente, firmar la subCA con la CARaiz, esto convertirá el CSR en un certificado válido, firmado con sha256 durante 5 años:

```
openssl x509 -req -in subCA/subCA.csr -CA raizCA/ca-raiz.crt -CAkey raizCA/ca-raiz.key -CAcreateserial -out subCA/subCA.crt -days 1825 -sha256 -extfile subCA/subCA.ext
Certificate request self-signature ok
subject=C = ES, ST = Granada, L = Granada, O = SWAP, CN = Sub_CA
```

- Ahora necesitaremos crear el certificado del servidor web:

- El primer paso es crear su clave privada:

```
openssl genrsa -out servidor/servidor.key 2048
```

- Una vez hecho esto crearemos un CSR que firmará subCA y lo convertirá en un certificado válido:

```
openssl req -new -key servidor/servidor.key -out servidor/servidor.csr -subj "/C=ES/ST=Granada/L=Granada/O=SWAP/CN=servidor.local"
```

- Crearemos un archivo de extensiones al igual que anteriormente para que se reconozca como válido:

```
# Define que este certificado no es de una CA
basicConstraints = CA:FALSE

# Permite firmar datos digitales y cifrar claves
keyUsage = critical, digitalSignature, keyEncipherment

# Específico para autenticación de servidores
extendedKeyUsage = serverAuth

# define nombres alternativos para el certificado
subjectAltName = @alt_names

# Nombre DNS asociado al servidor
[alt_names]
DNS.1 = servidor.local
```

- Por últimos firmamos el csr con subCA que hemos creado para que pase a ser un certificado válido:

```
openssl x509 -req -in servidor/servidor.csr -CA subCA/subCA.crt -CAkey subCA/subCA.key -CAcreateserial -out servidor/servidor.crt -days 825 -sha256 -extfile servidor/servidor.ext
Certificate request self-signature ok
subject=C = ES, ST = Granada, L = Granada, O = SWAP, CN = servidor.local
```

- El último a paso a realizar es crear la cadena de certificados, para esto concatenamos los diferentes certificados y los escribimos en un .pem, el objetivo de esto es que los usuarios puedan verificar la cadena completa(finalizando en CA-raiz):

```
cat servidor/servidor.crt subCA/subCA.crt raizCA/ca-raiz.crt > servidor/fullchain.pem
```

Vamos a verificar que la cadena de confianza es correcta:

```
openssl verify -CAfile <(cat subCA/subCA.crt raizCA/ca-raiz.crt) servidor/servidor.crt
servidor/servidor.crt: OK
```

Este último comando nos indica si servidor.crt está firmado correctamente. Para esto le mandamos los ficheros subCa.crt y ca-raiz.crt como cadena de confianza y le mandamos como certificado a verificar servidor.crt. Como podemos ver la salida nos indica OK en servidor.crt, esto significa que es un certificado válido y que la cadena de confianza es correcta.

Tareas Avanzadas - A2: Optimización de la configuración SSL en los servidores web

En esta tarea voy a mejorar la seguridad y el rendimiento de mis servidores apache. Para ello he consultado con gpt que es lo que lo hace más y menos seguro y he modificado mi .conf de la siguiente manera:

```
# Definimos el VirtualHost que escuchará en el puerto 443
<VirtualHost *:443>

    # Directorio raíz donde se encuentran los archivos que se van a servir
    DocumentRoot /var/www/html

    # Activamos SSL
    SSLEngine on

    # Indicamos donde se encuentra el SSL público
    SSLCertificateFile /etc/apache2/ssl/certificado_algarza04.crt

    # Indicamos donde se encuentra la clave privada
    SSLCertificateKeyFile /etc/apache2/ssl/certificado_algarza04.key

    # Protocolos seguros
    SSLProtocol -all +TLSv1.2 +TLSv1.3

    # Suites de cifrado fuertes y seguras
    SSLCipherSuite HIGH:!aNULL:!MD5:!RC4:!3DES:!NULL:!EXP:!LOW

    # Es el servidor el que elige los cifrados y no el cliente
    SSLHonorCipherOrder on
</VirtualHost>
```

- En primer lugar he añadido SSLProtocol -all +TLSv1.2 +TLSv1.3: esta orden desactiva todos los protocolos, una vez están todos desconectados activa TLS 1.2 y 1.3, esto se debe a que todos los demás están obsoletos y poseen vulnerabilidades, mientras que el 1.2 sigue siendo seguro y el 1.3 es más rápido y seguro.
- SSLCipherSuite HIGH:!aNULL:!MD5:!RC4:!3DES:!NULL:!EXP:!LOW: esta orden selecciona cifrados fuertes y todos los demás que aparecen los excluye:
 - !aNULL: Sin autenticación
 - !MD5: Algoritmo hash roto (colisiones posibles).
 - !RC4: Cifrado antiguo con múltiples fallos.
 - !3DES: Cifrado débil y lento (vulnerable a ataque Sweet32)
 - !NULL: Sin cifrado
 - !EXP: Cifrados exportables (desfasado)
 - !LOW: Cifrados de baja seguridad
- SSLHonorCipherOrder on: al ser el servidor el que elige el cifrado nos aseguramos que se va a elegir el mejor y el más seguro de los disponibles.

A continuación, procedí a probar la configuración SSL para asegurar que cumple con los requisitos de seguridad y rendimiento, para lo que utilicé SSL LABS. El resultado de esta fue varios avisos de seguridad,, aun así no afectan al resultado esperado. El primero fue que el certificado no estaba firmado por una autoridad de confianza, lo cual es esperado ya que es un certificado autofirmado hecho por mi, otro era el nombre CN, el cuál no coincide con el nombre del servidor, también era esperado debido a que son pruebas locales y por último me indicaba que no está habilitado HSTS, el cuál no está activado ya que no es necesario para la práctica.

Por otro lado, al analizar lo que nosotros buscábamos con la configuración (bloquear y habilitar ciertos servicios) podemos observar que esta ha sido exitoso:

```
Testing protocols via sockets except NPN+ALPN

SSLv2      not offered (OK)
SSLv3      not offered (OK)
TLS 1      not offered
TLS 1.1    not offered
TLS 1.2    offered (OK)
TLS 1.3    offered (OK): final
NPN/SPDY   not offered
ALPN/HTTP2 http/1.1 (offered)
```

```
Testing cipher categories

NULL ciphers (no encryption)          not offered (OK)
Anonymous NULL Ciphers (no authentication) not offered (OK)
Export ciphers (w/o ADH+NULL)         not offered (OK)
LOW: 64 Bit + DES, RC[2,4], MD5 (w/o export) not offered (OK)
Triple DES Ciphers / IDEA             not offered
Obsoleted CBC ciphers (AES, ARIA etc.) offered
Strong encryption (AEAD ciphers) with no FS offered (OK)
Forward Secrecy strong encryption (AEAD ciphers) offered (OK)
```

Como podemos ver solo están permitidos los protocolos que hemos habilitado y todos los suites de cifrado que hemos bloqueado no están disponibles.

Tareas Avanzadas - A3: Configuración de Caché de Sesiones SSL y Tickets de Sesión en el balanceador

En esta tarea voy a modificar el .conf de mi balanceador nginx para que guarde información sobre un usuario y de esta manera reducir el tiempo de conexión de este

Para esto me he informado con gpt sobre que parámetros debo añadir:

```
http {  
    # Habilita una caché compartida de 10MB para sesiones SSL  
    ssl_session_cache shared:SSL:10m;  
  
    # Establece un tiempo de expiración de 15 minutos para las sesiones SSL  
    ssl_session_timeout 15m;  
  
    # Activa los tickets de sesión SSL(reducir la carga del servidor)  
    ssl_session_tickets on;  
  
    # Define un grupo de servidores backend llamado "backend_algarza04".  
    # Serán los servidores controlados por el balanceador de carga  
    upstream backend_algarza04 {  
        server 192.168.10.2;
```

Los parámetros añadidos son los tres ssl que hay indicados dentro del http (al estar dentro del http se habilitarán para todos los servidores del backend_algarza04).

Para comprobar si se está reconectando vamos a utilizar `openssl s_client -connect localhost:8090 -reconnect`.

```
Post-Handshake New Session Ticket arrived:  
SSL-Session:  
    Protocol : TLSv1.3  
    Cipher   : TLS_AES_256_GCM_SHA384  
    Session-ID: AE792C410CD5DD953505CA96406905FA483AB3B6EE6AC3A49F2AE684798669BC  
    Session-ID-ctx:  
    Resumption PSK: 2390729168D062A28DFE3EF19786C2B1C356542DD0FF4B4C1D33861FF4A7502A67FDF640B84FE643C892135866927254  
    PSK identity: None  
    PSK identity hint: None  
    SRP username: None  
    TLS session ticket lifetime hint: 900 (seconds)  
    TLS session ticket:  
0000 - 2f 81 ce e0 9a 1e 47 7f-48 64 89 2e 0d 48 fe 70 /....G.Hd...H.p  
0010 - 45 1e 31 55 1f bd 2d ab-7e 28 a6 a0 f4 c8 02 1c E.1U...~(.....  
0020 - 69 f5 23 6e d4 8b 03 fd-94 a4 e3 cf 34 3f af ec i.#n.....4?...  
0030 - ce e2 0a e6 40 d5 1b c7-5b 89 46 b7 3e 87 8e 9d ....@...[.F.>...  
0040 - 41 f0 a4 d7 b6 26 ee f3-23 7b 01 7e 68 b9 b5 cb A...&...#{.-h...  
0050 - 59 57 0d 76 c6 22 ff 6d-c5 1f 7d 08 6a d0 a7 6c YW.v."m..).j..l  
0060 - 56 55 9d ab f6 6e 58 51-79 9b 96 92 e5 70 d2 1f VU...nXQy....p..  
0070 - e5 13 a0 03 60 a9 e7 7c-af 4d f7 76 45 48 18 6a ....'.|.M.vEH.j  
0080 - 62 d1 d1 98 99 d5 52 e7-59 51 d3 ac 30 0b 2a de b.....R.YQ..0.*.  
0090 - 01 29 79 95 04 a5 71 10-f5 29 b0 21 a3 f3 f3 7c .)y...q..)!...|  
00a0 - 1d 8e e4 30 50 8f 31 d5-cf 1c 7f 05 dc 24 fc 61 ...0P.1.....$.a  
00b0 - 06 29 7d 36 f5 c0 da bf-d7 8b 54 e9 1b 06 d2 2f .)}6.....T..../  
00c0 - a6 40 3b f5 9b a6 b4 53-c4 2a 0e d0 91 ce 17 90 .@;....S.*.....  
00d0 - 87 a4 a5 d1 32 b6 bc b8-4b bb 01 28 a1 0b 24 ce ....2...K..(...$.  
  
    Start Time: 1746188399  
    Timeout : 7200 (sec)  
    Verify return code: 0 (ok)  
    Extended master secret: no  
    Max Early Data: 0  
...  
read R BLOCK  
...
```

Como podemos ver en la salida está funcionando correctamente la reanudación de sesión, esto podemos verlo en la primera línea, que indica que el servidor mando un ticket tras el handshake con la información necesaria para que el usuario se reconecte en un futuro, además nos indica que tiene una vida útil de 900s(esto equivale a los 15 minutos que le indicamos en el .conf).

- A continuación voy a comprobar si el ticket está funcionando:
 - Realizo una primera conexión: `openssl s_client -connect localhost:8090 -tls1_3 -sess_out session.pem`.

```
SSL handshake has read 1627 bytes and written 305 bytes
Verification: OK
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
---
Post-Handshake New Session Ticket arrived:
SSL Session
```

- Realizo la segunda conexión: `openssl s_client -connect tu.dominio.com:443 -tls1_3 -sess_in session.pem` al contrario que el anterior este usa -sess_in (cargar una sesión previamente guardada) .

```
Reused, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Server public key is 2048 bit
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)
---
---
Post-Handshake New Session Ticket arrived:
```

```
TLS session ticket lifetime hint: 872 (seconds)
```

Como podemos observar en la primera conexión nos está indicando que esta es nueva, sin embargo en la segunda indica que se está reutilizando, además en la segunda conexión también marca en el tiempo restante del ticket (28 segundos menos de lo establecido), esto se debe a que ya han pasado 28 segundos desde que se creó el ticket.

Tareas Avanzadas - A4: Optimización de conexiones HTTPS y cifrado en el balanceador

Para optimizar la seguridad y el rendimiento he editado el archivo de configuración de nginx.

```
server {
    # escucha en el puerto 443, soporta SSL y HTTP/2
    listen 443 ssl http2;

    # Especifica la ubicación del certificado SSL y su clave privada
    ssl_certificate /etc/nginx/ssl/certificado_algarza04.crt;
    ssl_certificate_key /etc/nginx/ssl/certificado_algarza04.key;

    # Define los protocolos seguros habilitados (TLS 1.2 y TLS 1.3).
    ssl_protocols TLSv1.2 TLSv1.3;

    # Solo las suites más seguras y eficientes
    ssl_ciphers 'TLS_AES_256_GCM_SHA384:TLS_AES_128_GCM_SHA256:TLS_CHACHA20_POLY1305_SHA256:
                ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:
                ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384';

    # Prioridad a las suites definidas por el servidor
    ssl_prefer_server_ciphers on;
}
```

(a pesar

de que aquí tenga los ssl_ciphers en diferentes líneas en mi archivo están en una ya que daba error)

A continuación se pueden ver los parámetros que le he añadido:

- Añadir http2 al listen permite mejorar la eficiencia y el rendimiento al permitir enviar varias solicitudes en una misma conexión.
- ssl_protocols TLSv1.2 TLSv1.3 esto habilita ambos protocolos (como hemos visto en el ejercicio anterior son los más seguros que hay) y desactiva los inseguros.
- ssl_ciphers: para este apartado le pregunté a gpt cuáles son las más eficientes y estas son las que añadí (todas soportan PFS (Perfect Forward Secrecy)):
 - TLS_AES_256_GCM_SHA384: Cifrado muy fuerte y seguro.
 - TLS_AES_128_GCM_SHA256: Cifrado similar al anterior pero más rápido.
 - TLS_CHACHA20_POLY1305_SHA256: Cifrado rápido para móviles o dispositivos sin buen hardware.
 - ECDHE-ECDSA-AES128-GCM-SHA256: Cifrado seguro que usa curvas elípticas y AES-128 para mayor velocidad.
 - ECDHE-RSA-AES128-GCM-SHA256: Similar al anterior, pero usa RSA para la autenticación.
 - ECDHE-ECDSA-AES256-GCM-SHA384: Versión más fuerte del 4, con AES-256.
 - ECDHE-RSA-AES256-GCM-SHA384: Igual al anterior, pero con RSA para la autenticación.
- ssl_prefer_server_ciphers on: al ser el servidor el que elige las suites permite elegir los certificados más seguros.

Por último voy a comprobar que la implementación haya sido correcta.

- Proxy:

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P3$ curl -k -I https://localhost:8090
HTTP/2 200
server: nginx/1.27.5
date: Fri, 02 May 2025 16:53:21 GMT
content-type: text/html; charset=UTF-8
```

- SSL/TLS:

```
CONNECTED(00000003)
Can't use SSL_get_servername
depth=0 C = ES, ST = Granada, L = Granada, O = SWAP, OU = Pr\C3\83\C2\A1ctica 3, CN = \C3\83\C2\81lvaro Garc\C3\83\C2\Ada Zafra, emailAddress = algarza04@correo.ugr.es
verify return:1
---
Certificate chain
 0 s:C = ES, ST = Granada, L = Granada, O = SWAP, OU = Pr\C3\83\C2\A1ctica 3, CN = \C3\83\C2\81lvaro Garc\C3\83\C2\Ada Zafra, emailAddress = algarza04@correo.ugr.es
  i:C = ES, ST = Granada, L = Granada, O = SWAP, OU = Pr\C3\83\C2\A1ctica 3, CN = \C3\83\C2\81lvaro Garc\C3\83\C2\Ada Zafra, emailAddress = algarza04@correo.ugr.es
  a:PKCS7: rsaEncryption, 2048 (bit); sigalg: RSA-SHA256
  v:NotBefore: Apr 28 08:28:22 2025 GMT; NotAfter: Apr 28 08:28:22 2026 GMT
```

```
Post-Handshake New Session Ticket arrived:
SSL-Session:
    Protocol : TLSv1.3
    Cipher   : TLS_AES_256_GCM_SHA384
    Session-ID: AE0842621BB8DB5F43153920A7206C674601BD6DED6E5785DF
    Session-ID-ctx:
    Resumption PSK: A9ABD4555FC02CD6DD2078665F1A6D575D2C73D8874937
    3C8F853D69E5E6738FA0A3
```

Tras examinar las salidas de ambos comandos, podemos decir que esta bien implementado y que está utilizando de manera correcta las herramientas añadidas, como puede ser HTTP/2 y TLS_AES_256...

ANÁLISIS PROPUESTA IA

enlace al chat: <https://chatgpt.com/share/6814fabd-6268-800d-9704-c0a380bde49b>

Para finalizar esta práctica voy a hablar un poco sobre cómo he utilizado la IA a lo largo de esta práctica.

Al igual que en las dos prácticas anteriores el uso de esta en las tareas básicas ha sido simplemente para saber que pasos seguir en algún apartado (creación de un certificado) y solucionar algunos errores (error en el dockerfile al hacer el COPY ya que no sabía que no podía llamar a un archivo que tuviese una ruta diferente a la localización actual).

Por otro lado en las tareas avanzadas ha sido una herramienta muy útil, me ha servido para saber que pasos seguir en cada uno de los certificados a crear y para saber que comandos había que utilizar para comprobar si estaban correctamente creados, también la he utilizado para poder justificar el uso de protocolos, es decir, le pregunté que realizaba cada uno de los protocolos para así poder justificar su uso.

A lo largo de esta práctica no he encontrado información de gpt que no me haya podido ser útil o que no haya sido exactamente lo que le he pedido, simplemente algunos pequeños errores en alguna configuración o Dockerfile que posteriormente me he resuelto.

Al igual que he mencionado en las prácticas anteriores, esta es una herramienta muy útil que no se debe utilizar mandando un texto y copiando tal cuál su salida, si no que debemos pedir lo que necesitamos y saber entender la información que nos está dando, y de esta manera usar lo que vamos a necesitar y lo que no. Si conseguimos realizar lo último que he dicho de una manera eficiente se convertirá en la mejor herramienta para desarrollar cualquier tipo de página, app, servidor..., incluso cuándo no tengamos una base sólida sobre ello.